An example
of
Projecting global sea-level
using
FACTS

Intro to Installing FACTS

- What you will run today is a containerized (Docker), summer-school-specific build of FACTS and is **NOT the general distribution**.
- Runs on standard laptop, workstations and Desktops running macOS, Windows, and Linux.
- HPC setup is very different.
- Watch RAM and disk space at all times running the container
  more samples or locations means much larger files.
  Big experiments will crash a laptop. Start small and scale up.

Git brings the recipe

source code + Dockerfile + helper scripts

Use the recipe to build the kitchen
(the container)
then cook (run FACTS) the same way for everyone.

Todays Prerequisites:

Docker installed and running, **~30 GB free disk space**.

You can also install on a **flashdrive**.

pkjr002

Before you Install:: READ

Software

- Docker Desktop (running)
- git + SSH key for GitHub
- Terminal (bash / zsh / WSL , IDE)

Hardware

- ~30 GB free disk
- ≥ 8 GB RAM (16 GB recommended)
- macOS / Windows / Linux
- Min 2 processors

Reference

- FACTS Quick Start
- Git code
- This slide deck

⚠ The HPC setup is structurally different. Don't try to lift this Docker workflow onto a cluster on your own — talk to the dev team.

kmccusker@impactlab.org

Kelly McCusker

Associate Director,

Rhodium Group

emarshall@rhg.com

Emma Marshall

Research

Software Developer,

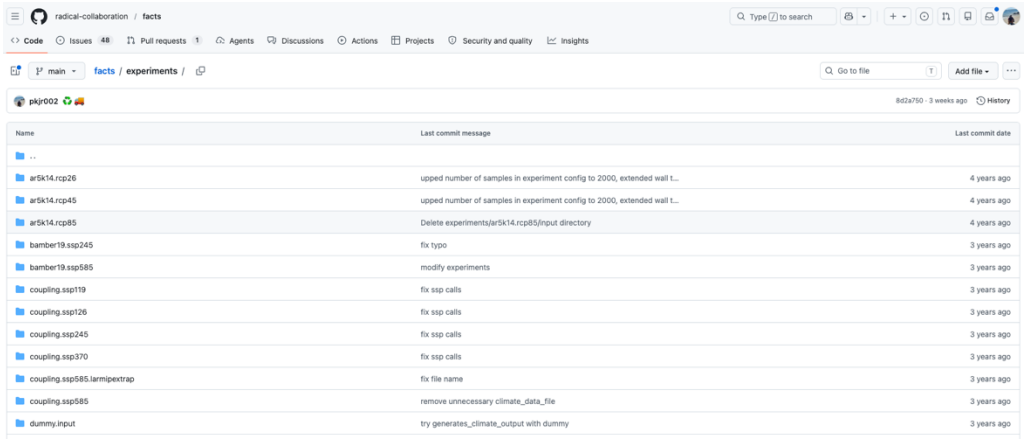
Rhodium Group

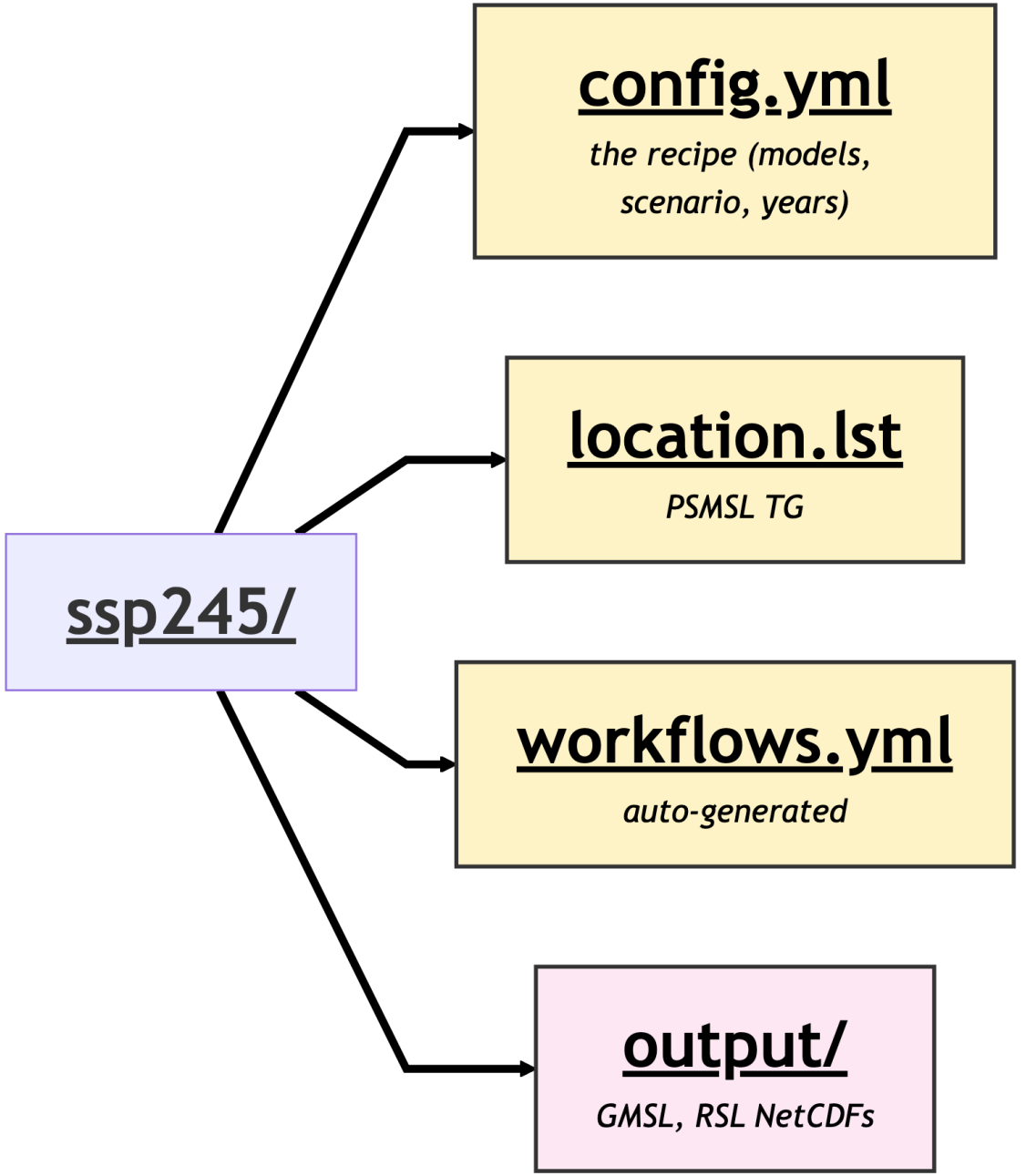
Install FACTS

Use the guide Provided to

FACTS experiment

- A FACTS experiment (**SSP245**) is just a directory.
  - Everything that defines the projection lives inside it.
- Run it by pointing runFACTS.py at the directory.





FACTS Run

```
$ bash
submit_docker_ssisls_experiment.sh
ssp245
```

- I have configured FACTS to run with a submit script
- This is a very refined version of running the model.

RADICAL Thinking

Research in Advanced DIstributed Cyber- infrastructure and Applications Laboratory. Working at the intersection of computing and science to advance cyberinfrastructure tools.

https://github.com/radical-cybertools

FACTS Experiment

Steps

Climate

One module is run during the climate step.

Within each **module** in an **component of sealevel**, stages are executed sequentially.

Stages: preprocess, fit, project, postprocess

Sea-level

module/pipeline ... ... | module/pipeline ... ... | module/pipeline ... ... | module/pipeline ... ... | module/pipeline ... ... | ...

Multiple module representing different sea-level components can be run during sea-level step.

Workflow: Combination of modules run in sea-level step to be summed into a total projection of sea-level change.

Wf1: + Wf2: + Wf3: +

Totaling

Emma Marshall

Research Software Developer, Rhodium Group

config.yml

config.yml

```yaml
global-options:
    nsamps: 2000
    scenario: ssp585
    pyear_start: 2020
    pyear_end:   2150
    pyear_step:  10
    baseyear:    2005

climate_step:             # produces GSAT / OHC for the SLR modules
    temperature: { module_set: fair, module: temperature,
                   generates_climate_output: true }

sealevel_step:            # the projection modules (the big one)
    GrIS1f:    { module_set: FittedISMIP, module: GrIS,        include_in_workflow: [wf1f, wf2f, wf3f] }
    deconto21: { module_set: deconto21,   module: AIS,         include_in_workflow: [wf3e, wf3f] }
    larmip:    { module_set: larmip,      module: AIS,         include_in_workflow: [wf2e, wf2f] }
    ocean:     { module_set: tlm,         module: sterodynamics, include_in_workflow: [wf1f, wf1e, wf2e, ...] }
    ...

totaling_step: { facts/total · loop over workflows + scales }
esl_step:      { extremesealevel/pointsoverthreshold · loop over workflows }
```

workflows.yml — auto-generated

You can think of each workflow as a total sea-level from summing over a specific combination of modules

workflows.yml (auto-generated)

```
 wf1e:
    global:
    - global/coupling.ssp585.emuAIS.emulandice.AIS_globalsl.nc
    - global/coupling.ssp585.emuGrIS.emulandice.GrIS_globalsl.nc
    - global/coupling.ssp585.emuglaciers.emulandice.glaciers_globalsl.nc
    - global/coupling.ssp585.ocean.tlm.sterodynamics_globalsl.nc
    - global/coupling.ssp585.lws.ssp.landwaterstorage_globalsl.nc
    local: []
    options:
      pyear_end: 2100

 wf1f:
    global:
    - global/coupling.ssp585.GrIS1f.FittedISMIP.GrIS_GIS_globalsl.nc
    - global/coupling.ssp585.ar5glaciers.ipccar5.glaciers_globalsl.nc
    ...
```

What lands in output/ (NetCDFs)

```python
# Canonical NetCDF schema for every output:
sea_level_change : float32  (samples=2000, years=14, locations=1)   units = mm
lat              : float64  (locations,)
lon              : float64  (locations,)
years            : int64    (years,)            # 2020, 2030, ..., 2150
locations        : int64    (locations,)        # PSMSL ids from location.lst

# Plotting at New York:
import xarray as xr
ds = xr.open_dataset('output/coupling.ssp585.total.workflow.wf3f.local.nc')
samples = ds['sea_level_change'].isel(locations=0)
q50, q05, q95 = samples.quantile([0.5, 0.05, 0.95], dim='samples')
```

... welcome to FACTS

The EnD